

Complete Machine Learning Workflow: From Zero to Deployment

Table of Contents

1. Problem Definition
 2. Data Collection
 3. Exploratory Data Analysis (EDA)
 4. Data Cleaning
 5. Feature Engineering
 6. Data Splitting
 7. Model Selection
 8. Model Training
 9. Model Evaluation
 10. Model Optimization
 11. Model Deployment
 12. Monitoring & Maintenance
-

1. Problem Definition

WHY: Why do we define the problem first?

Imagine you're a doctor. Before prescribing medicine, you need to know: What's wrong? How serious is it? What outcome do we want?

Machine Learning is the same. Without a clear problem definition:

- You might build something nobody needs
- You won't know if your solution actually works
- You'll waste time on the wrong approach

HOW: How do we define the problem?

We answer these questions:

1. What exactly are we trying to predict or accomplish?
2. Why does this matter? (Business value)
3. How will we measure success?
4. What data do we have or can we get?
5. What type of ML problem is this?

DO IT: Real Example

Scenario: A bank wants to reduce losses from customers who don't repay loans.

PROBLEM STATEMENT:

What: Predict whether a loan applicant will default (fail to repay)

Why: Losing \$10M annually to bad loans. Reducing this by 30% saves \$3M/year

Success Metric: Catch 70% of defaulters while approving 80% of good customers

Data Available: 50,000 past loan applications with outcomes

Problem Type: Binary Classification (Yes/No prediction)

SPECIFIC GOALS:

- Primary: Precision of 75% (when we say "will default", be right 75% of time)
- Secondary: Recall of 70% (catch 70% of actual defaulters)
- Constraint: Process must complete in < 2 seconds for real-time decisions

Why these specific numbers?

- 75% precision: Bank can handle 25% false alarms
- 70% recall: Willing to miss 30% of defaulters to not reject too many good customers
- 2 seconds: Customers won't wait longer during application

2. Data Collection

WHY: Why is data collection critical?

Your model can only learn patterns from the data you give it. It's like teaching someone to recognize animals:

- Show them only cats → They can't identify dogs
- Show them blurry photos → They'll make mistakes on clear photos
- Show them 10 photos → They won't learn much
- Show them 10,000 varied photos → They'll learn well

Garbage in = Garbage out

HOW: How do we collect data?

Sources of data:

1. **Internal databases** (company records)
2. **APIs** (pulling from external services)
3. **Files** (CSV, Excel, JSON)
4. **Web scraping** (extracting from websites)
5. **Surveys/Forms** (collecting new data)
6. **IoT sensors** (real-time measurements)

DO IT: Real Example

For our loan default prediction:

python

Installing required libraries

In terminal: pip install pandas numpy

```
import pandas as pd
```

```
import numpy as np
```

Method 1: Reading from CSV file

Imagine the bank has a file called 'loan_applications.csv'

```
data = pd.read_csv('loan_applications.csv')
```

Method 2: Connecting to a database

(More common in real companies)

```
from sqlalchemy import create_engine
```

Create connection to database

```
engine = create_engine('postgresql://user:password@localhost:5432/bank_db')
```

SQL query to get data

```
query = """
```

```
SELECT
```

```
    application_id,
```

```
    applicant_age,
```

```
    annual_income,
```

```
    loan_amount,
```

```
    credit_score,
```

```
    employment_years,
```

```
    has_mortgage,
```

```
    has_car_loan,
```

```
    defaulted -- This is what we want to predict
```

```
FROM loan_applications
```

```
WHERE application_date >= '2020-01-01'
```

```
"""
```

```
data = pd.read_sql(query, engine)
```

Method 3: Creating sample data for learning

(Since you don't have a real bank database)

```
np.random.seed(42) # For reproducibility
```

```
sample_data = {
```

```
    'application_id': range(1, 1001),
```

```
    'applicant_age': np.random.randint(18, 70, 1000),
```

```
    'annual_income': np.random.randint(20000, 200000, 1000),
```

```
    'loan_amount': np.random.randint(5000, 50000, 1000),
```

```
    'credit_score': np.random.randint(300, 850, 1000),
```

```
    'employment_years': np.random.randint(0, 40, 1000),
```

```
    'has_mortgage': np.random.choice([0, 1], 1000),
```

```
    'has_car_loan': np.random.choice([0, 1], 1000),
```

```

'has_car_loan': np.random.choice([0, 1], 1000),
'defaulted': np.random.choice([0, 1], 1000, p=[0.85, 0.15]) # 15% default rate
}

data = pd.DataFrame(sample_data)

# Save for later use
data.to_csv('loan_data.csv', index=False)

print(f>Data collected: {len(data)} records")
print(f>Columns: {list(data.columns)}")

```

Output:

```

Data collected: 1000 records
Columns: ['application_id', 'applicant_age', 'annual_income', 'loan_amount',
'credit_score', 'employment_years', 'has_mortgage', 'has_car_loan', 'defaulted']

```

What just happened?

- We collected 1,000 loan applications
 - Each row = one person's loan application
 - Each column = one piece of information about that person
 - `defaulted` column = whether they failed to repay (1) or paid back (0)
-

3. Exploratory Data Analysis (EDA)

WHY: Why explore the data?

Imagine receiving a 1000-page book in a foreign language. Before translating it, you'd want to:

- Check if pages are missing
- See if there are pictures or just text
- Notice any patterns or repeated sections

EDA does this for data. We discover:

- What the data actually looks like
- Problems we need to fix
- Patterns that might help predictions
- Which features might be useful

HOW: How do we explore data?

Five key activities:

1. **Basic inspection** - Shape, types, first look
2. **Statistical summaries** - Mean, median, min, max
3. **Missing values check** - What's incomplete?
4. **Distribution analysis** - How values are spread
5. **Relationship exploration** - What correlates with what?

DO IT: Real Example

```
python
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Load our data
```

```
data = pd.read_csv('loan_data.csv')
```

```
# =====
```

```
# STEP 1: Basic Inspection
```

```
# =====
```

```
print("=" * 50)
```

```
print("BASIC INFORMATION")
```

```
print("=" * 50)
```

```
# How many rows and columns?
```

```
print(f"Dataset shape: {data.shape}")
```

```
print(f" → {data.shape[0]} applications (rows)")
```

```
print(f" → {data.shape[1]} features (columns)\n")
```

```
# What type is each column?
```

```
print("Data types:")
```

```
print(data.dtypes)
```

```
print()
```

```
# First few rows
```

```
print("First 5 applications:")
```

```
print(data.head())
```

```
print()
```

```
# =====
```

```
# STEP 2: Statistical Summary
```

```
# =====
```

```
print("=" * 50)
```

```
print("STATISTICAL SUMMARY")
```

```
print("=" * 50)
```

```
print(data.describe())
```

```
print()
```

```
# What does this tell us?
```

```
print("INSIGHTS FROM SUMMARY:")
```

```
print(f"• Average age: {data['applicant_age'].mean():.1f} years")
```

```
print(f"• Average income: ${data['annual_income'].mean():,.0f}")
```

```
print(f"• Average loan: ${data['loan_amount'].mean():,.0f}")
```

```
print(f"• Total number of applications: {data.shape[0]}")
```

```

print(f"• Default rate: {data['defaulted'].mean():.1%}")
print()

# =====
# STEP 3: Missing Values Check
# =====

print("=" * 50)
print("MISSING VALUES")
print("=" * 50)

missing = data.isnull().sum()
missing_percent = (missing / len(data)) * 100

missing_df = pd.DataFrame({
    'Missing Count': missing,
    'Percentage': missing_percent
})

print(missing_df[missing_df['Missing Count'] > 0])

if missing.sum() == 0:
    print("✓ No missing values found!")
print()

# =====
# STEP 4: Distribution Analysis
# =====

print("=" * 50)
print("DISTRIBUTION ANALYSIS")
print("=" * 50)

# Check target variable balance
default_counts = data['defaulted'].value_counts()
print("Default Distribution:")
print(f" Paid back (0): {default_counts[0]} ({default_counts[0]/len(data):.1%}")
print(f" Defaulted (1): {default_counts[1]} ({default_counts[1]/len(data):.1%}")
print()

# This is CRITICAL to check!
if default_counts[1] / len(data) < 0.10:
    print("⚠ WARNING: Imbalanced dataset! Only",
          f"{default_counts[1]/len(data):.1%} are defaults.")
    print(" We'll need special techniques to handle this.")
print()

# =====
# STEP 5: Relationship Exploration
# =====

```



```
print("=" * 50)
print("CORRELATIONS WITH DEFAULT")
print("=" * 50)

# Calculate correlation with target
correlations = data.corr()['defaulted'].sort_values(ascending=False)
print(correlations)
print()

print("INTERPRETATION:")
print("• Positive correlation: Higher value → More likely to default")
print("• Negative correlation: Higher value → Less likely to default")
print("• Close to 0: Little/no relationship")
```

Sample Output:

BASIC INFORMATION

Dataset shape: (1000, 9)

→ 1000 applications (rows)

→ 9 features (columns)

Data types:

application_id	int64
applicant_age	int64
annual_income	int64
loan_amount	int64
credit_score	int64
employment_years	int64
has_mortgage	int64
has_car_loan	int64
defaulted	int64

STATISTICAL SUMMARY

	application_id	applicant_age	annual_income	loan_amount	credit_score
count	1000.00	1000.00	1000.00	1000.00	1000.00
mean	500.50	43.85	109420.00	27345.00	574.50
std	288.82	14.93	51850.23	12950.48	158.89
min	1.00	18.00	20050.00	5020.00	301.00
25%	250.75	31.00	65125.00	16230.00	438.00
50%	500.50	44.00	109500.00	27315.00	574.50
75%	750.25	57.00	154000.00	38475.00	711.00
max	1000.00	69.00	199950.00	49990.00	849.00

MISSING VALUES

✓ No missing values found!

DEFAULT DISTRIBUTION

Paid back (0): 850 (85.0%)

Defaulted (1): 150 (15.0%)

What did we learn?

1. **Dataset size:** 1,000 applications - might need more data for complex models
 2. **Age range:** 18-69 years - covers full adult spectrum
 3. **Income:** \$20k-\$200k - wide range, will need scaling
 4. **Default rate:** 15% - somewhat imbalanced, but manageable
 5. **No missing values:** Great! No cleaning needed yet
-

4. Data Cleaning

WHY: Why clean data?

Imagine learning to recognize handwriting, but:

- Some examples have coffee stains (missing data)
- Some letters are written twice (duplicates)
- Some samples are actually random scribbles (outliers)

Your learning would be poor! Machine learning models have the same problem. Dirty data leads to:

- Incorrect patterns learned
- Poor predictions
- Model crashes or errors

HOW: How do we clean data?

Common cleaning tasks:

1. **Handle missing values** - Fill or remove
2. **Remove duplicates** - Keep only unique records
3. **Fix data types** - Ensure columns are correct format
4. **Handle outliers** - Deal with extreme values
5. **Standardize formats** - Consistent naming, casing

DO IT: Real Example

Let's create messier data first, then clean it:

python

```
import pandas as pd
import numpy as np

# Load our clean data
data = pd.read_csv('loan_data.csv')

# =====
# CREATING MESSY DATA (for demonstration)
# =====

# Introduce missing values
data_messy = data.copy()
data_messy.loc[np.random.choice(data_messy.index, 50), 'annual_income'] = np.nan
data_messy.loc[np.random.choice(data_messy.index, 30), 'credit_score'] = np.nan

# Add duplicates
duplicates = data_messy.sample(20)
data_messy = pd.concat([data_messy, duplicates], ignore_index=True)

# Add outliers
data_messy.loc[data_messy.sample(5).index, 'annual_income'] = 5000000 # $5M salary
data_messy.loc[data_messy.sample(5).index, 'applicant_age'] = 150 # 150 years old!

print("CREATED MESSY DATA:")
print(f"Shape: {data_messy.shape}")
print(f"Missing income values: {data_messy['annual_income'].isnull().sum()}")
print(f"Missing credit score values: {data_messy['credit_score'].isnull().sum()}")
print(f"Total rows (including duplicates): {len(data_messy)}")
print()

# =====
# CLEANING STEP 1: Remove Duplicates
# =====

print("=" * 50)
print("STEP 1: REMOVING DUPLICATES")
print("=" * 50)

print(f"Before: {len(data_messy)} rows")

# Remove duplicates based on all columns
data_clean = data_messy.drop_duplicates()

print(f"After: {len(data_clean)} rows")
print(f"Removed: {len(data_messy) - len(data_clean)} duplicate rows")
print()
```

```

# =====

# CLEANING STEP 2: Handle Missing Values
# =====

print("=" * 50)
print("STEP 2: HANDLING MISSING VALUES")
print("=" * 50)

missing_before = data_clean.isnull().sum()
print("Missing values before cleaning:")
print(missing_before[missing_before > 0])
print()

# Strategy depends on the column:

# For annual_income: Fill with median (not affected by outliers)
income_median = data_clean['annual_income'].median()
data_clean['annual_income'].fillna(income_median, inplace=True)
print(f"✓ Filled missing income with median: ${income_median:,.0f}")

# For credit_score: Fill with median
credit_median = data_clean['credit_score'].median()
data_clean['credit_score'].fillna(credit_median, inplace=True)
print(f"✓ Filled missing credit score with median: {credit_median:.0f}")

# Alternative strategies:
# - fillna(mean) - use average
# - fillna(0) - use zero
# - fillna(method='ffill') - use previous row's value
# - dropna() - remove rows with missing values

print(f"\nMissing values after cleaning: {data_clean.isnull().sum().sum()}")
print()

# =====

# CLEANING STEP 3: Handle Outliers
# =====

print("=" * 50)
print("STEP 3: HANDLING OUTLIERS")
print("=" * 50)

# Method: Interquartile Range (IQR)
# This identifies values that are too far from the median

def detect_outliers_iqr(data, column):
    """
    Detect outliers using IQR method

    What is IQR?

```

What is IQR:

- Q1 (25th percentile): 25% of data is below this
- Q3 (75th percentile): 75% of data is below this
- IQR = Q3 - Q1 (the middle 50% range)
- Outliers: Values beyond $Q1 - 1.5 * IQR$ or $Q3 + 1.5 * IQR$

"""

```
Q1 = data[column].quantile(0.25)
```

```
Q3 = data[column].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
lower_bound = Q1 - 1.5 * IQR
```

```
upper_bound = Q3 + 1.5 * IQR
```

```
outliers = data[(data[column] < lower_bound) | (data[column] > upper_bound)]
```

```
return outliers, lower_bound, upper_bound
```

```
# Check age outliers
```

```
age_outliers, age_lower, age_upper = detect_outliers_iqr(data_clean, 'applicant_age')
```

```
print(f"Age outliers detected: {len(age_outliers)}")
```

```
print(f" Valid range: {age_lower:.1f} to {age_upper:.1f} years")
```

```
if len(age_outliers) > 0:
```

```
    print(f" Outlier values: {age_outliers['applicant_age'].values}")
```

```
    # Remove impossible ages
```

```
    data_clean = data_clean[data_clean['applicant_age'] <= 100]
```

```
    print(f" ✓ Removed ages > 100 years")
```

```
print()
```

```
# Check income outliers
```

```
income_outliers, income_lower, income_upper = detect_outliers_iqr(data_clean, 'annual_income')
```

```
print(f"Income outliers detected: {len(income_outliers)}")
```

```
print(f" Valid range: ${income_lower:,.0f} to ${income_upper:,.0f}")
```

```
if len(income_outliers) > 0:
```

```
    print(f" Outlier incomes: {income_outliers['annual_income'].values[:5]}") # Show first 5
```

```
    # Cap extreme values instead of removing
```

```
    data_clean['annual_income'] = data_clean['annual_income'].clip(upper=500000)
```

```
    print(f" ✓ Capped income at $500,000")
```

```
print()
```

```
# =====
```

```
# CLEANING STEP 4: Data Type Validation
```

```
# =====
```

```
print("=" * 50)
```

```
print("STEP 4: DATA TYPE VALIDATION")
```

```
print("=" * 50)
```

```

print("Current data types:")
print(data_clean.dtypes)
print()

# Ensure binary columns are 0/1
for col in ['has_mortgage', 'has_car_loan', 'defaulted']:
    unique_values = data_clean[col].unique()
    if not set(unique_values).issubset({0, 1}):
        print(f"⚠ Warning: {col} has values: {unique_values}")
    else:
        print(f"✓ {col} is valid binary")

print()

# =====
# CLEANING SUMMARY
# =====

print("=" * 50)
print("CLEANING SUMMARY")
print("=" * 50)

print(f"Original rows: {len(data_messy)}")
print(f"Final rows: {len(data_clean)}")
print(f"Rows removed: {len(data_messy) - len(data_clean)}")
print(f"Missing values: {data_clean.isnull().sum().sum()}")
print(f"\n✓ Data is now clean and ready for modeling!")

# Save cleaned data
data_clean.to_csv('loan_data_cleaned.csv', index=False)
print("\n✓ Saved to 'loan_data_cleaned.csv'")

```

Sample Output:

CREATED MESSY DATA:

Shape: (1020, 9)

Missing income values: 50

Missing credit score values: 30

Total rows (including duplicates): 1020

STEP 1: REMOVING DUPLICATES

Before: 1020 rows

After: 1000 rows

Removed: 20 duplicate rows

STEP 2: HANDLING MISSING VALUES

Missing values before cleaning:

annual_income 50

credit_score 30

dtype: int64

✓ Filled missing income with median: \$109,500

✓ Filled missing credit score with median: 575

Missing values after cleaning: 0

STEP 3: HANDLING OUTLIERS

Age outliers detected: 5

Valid range: -8.0 to 95.0 years

Outlier values: [150 150 150 150 150]

✓ Removed ages > 100 years

Income outliers detected: 5

Valid range: \$-49,312 to \$243,732

Outlier incomes: [5000000 5000000 5000000]

✓ Capped income at \$500,000

Key Decisions Made:

1. **Duplicates:** Removed completely (no reason to have same person twice)
2. **Missing income:** Filled with median (robust to outliers)
3. **Missing credit score:** Filled with median
4. **Age outliers:** Removed (150 years = data error)
5. **Income outliers:** Capped (some people do earn \$500k+, but \$5M likely error)

5. Feature Engineering

WHY: Why engineer features?

Imagine teaching someone to judge house prices. You could tell them:

- Bedrooms: 3
- Bathrooms: 2
- Size: 2000 sq ft

OR you could help them by calculating:

- Price per square foot
- Bedroom to bathroom ratio
- Is it bigger than average?

The second approach gives them better information to make judgments. Feature engineering does this for ML models - it creates new, more useful information from existing data.

HOW: How do we engineer features?

Common techniques:

1. **Creating ratios** - Divide one feature by another
2. **Binning** - Group continuous values into categories
3. **Combining features** - Multiply or add features together
4. **Extracting components** - Pull out parts (like month from date)
5. **Encoding categories** - Convert text to numbers
6. **Scaling** - Adjust ranges to be comparable

DO IT: Real Example

```
python
```

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, LabelEncoder
```

```
# Load cleaned data
```

```
data = pd.read_csv('loan_data_cleaned.csv')
```

```
print("=" * 50)
print("ORIGINAL FEATURES")
print("=" * 50)
print(f"Starting with {data.shape[1]} features:")
print(list(data.columns))
print()
```

```
# =====
# TECHNIQUE 1: Creating Ratios
# =====
```

```
print("=" * 50)
print("TECHNIQUE 1: CREATING RATIOS")
print("=" * 50)
```

```
# Debt-to-Income Ratio (very important in lending!)
```

```
# Higher ratio = more risky borrower
```

```
data['debt_to_income_ratio'] = data['loan_amount'] / data['annual_income']
```

```
print("✓ Created: debt_to_income_ratio")
print(f" Example: Loan $30,000 / Income $100,000 = {30000/100000:.2f} (30%)")
print(f" Range: {data['debt_to_income_ratio'].min():.2%} to {data['debt_to_income_ratio'].max():.2%}")
print()
```

```
# Loan-to-Income Ratio (another view of the same concept)
```

```
data['loan_to_income'] = data['loan_amount'] / (data['annual_income'] / 12)
```

```
print("✓ Created: loan_to_income")
print(f" This shows: loan as multiple of monthly income")
print(f" Range: {data['loan_to_income'].min():.2f}x to {data['loan_to_income'].max():.2f}x")
print()
```

```
# =====
# TECHNIQUE 2: Binning Continuous Variables
# =====
```

```
print("=" * 50)
print("TECHNIQUE 2: BINNING")
print("=" * 50)
```

Age groups (instead of exact age)

Sometimes categories work better than exact numbers

```
age_bins = [0, 25, 35, 45, 55, 100]
```

```
age_labels = ['18-25', '26-35', '36-45', '46-55', '55+']
```

```
data['age_group'] = pd.cut(data['applicant_age'],  
                           bins=age_bins,  
                           labels=age_labels)
```

```
print("✓ Created: age_group")
```

```
print(" Age ranges:")
```

```
for label in age_labels:
```

```
    count = (data['age_group'] == label).sum()
```

```
    print(f" {label}: {count} people")
```

```
print()
```

Income brackets

```
income_bins = [0, 30000, 50000, 75000, 100000, float('inf')]
```

```
income_labels = ['Low', 'Lower-Mid', 'Mid', 'Upper-Mid', 'High']
```

```
data['income_bracket'] = pd.cut(data['annual_income'],  
                                bins=income_bins,  
                                labels=income_labels)
```

```
print("✓ Created: income_bracket")
```

```
print(" Income distribution:")
```

```
print(data['income_bracket'].value_counts().sort_index())
```

```
print()
```

```
# =====
```

TECHNIQUE 3: Combining Features

```
# =====
```

```
print("=" * 50)
```

```
print("TECHNIQUE 3: COMBINING FEATURES")
```

```
print("=" * 50)
```

Total debt burden (combining existing debt indicators)

```
data['total_existing_debt'] = data['has_mortgage'] + data['has_car_loan']
```

```
print("✓ Created: total_existing_debt")
```

```
print(" Possible values:")
```

```
print(f" 0 (no debts): {(data['total_existing_debt']==0).sum()} people")
```

```
print(f" 1 (one debt): {(data['total_existing_debt']==1).sum()} people")
```

```
print(f" 2 (both debts): {(data['total_existing_debt']==2).sum()} people")
```

```
print()
```

Financial stability score

Higher credit score + more employment years = more stable

```
data['stability_score'] = (data['credit_score'] / 850) + (data['employment_years'] / 40)
```

```
data['stability_score'] = data['credit_score'] / 100 + (data['employment_years'] / 10)

print("✓ Created: stability_score")
print(" Combines credit score and employment length")
print(f" Range: {data['stability_score'].min():.2f} to {data['stability_score'].max():.2f}")
print()
```

```
# =====
# TECHNIQUE 4: Polynomial Features
# =====
```

```
print("=" * 50)
print("TECHNIQUE 4: POLYNOMIAL FEATURES")
print("=" * 50)
```

```
# Sometimes relationships aren't linear
# Example: Risk might increase exponentially with age
```

```
data['age_squared'] = data['applicant_age'] ** 2
print("✓ Created: age_squared")
print(" Captures non-linear age effects")
print()
```

```
# Income-Credit Score interaction
# High income + high credit = very safe
# Low income + low credit = very risky
data['income_credit_interaction'] = data['annual_income'] * data['credit_score']
```

```
print("✓ Created: income_credit_interaction")
print(" Captures combined effect of income and creditworthiness")
print()
```

```
# =====
# TECHNIQUE 5: Boolean Features
# =====
```

```
print("=" * 50)
print("TECHNIQUE 5: BOOLEAN FEATURES")
print("=" * 50)
```

```
# Is this a risky loan? (high debt-to-income)
data['high_debt_ratio'] = (data['debt_to_income_ratio'] > 0.4).astype(int)
```

```
print("✓ Created: high_debt_ratio")
print(f" High risk (>40% DTI): {data['high_debt_ratio'].sum()} applications")
print()
```

```
# Is applicant young?
data['is_young'] = (data['applicant_age'] < 30).astype(int)
```

```

print("✓ Created: is_young")
print(f" Young applicants (<30): {data['is_young'].sum()}")
print()

# High credit score flag
data['excellent_credit'] = (data['credit_score'] >= 750).astype(int)

print("✓ Created: excellent_credit")
print(f" Excellent credit (≥ 750): {data['excellent_credit'].sum()}")
print()

# =====
# TECHNIQUE 6: Encoding Categorical Variables
# =====

print("=" * 50)
print("TECHNIQUE 6: ENCODING CATEGORIES")
print("=" * 50)

# Our age_group and income_bracket are categories (text)
# ML models need numbers!

# Method A: Label Encoding (for ordered categories)
print("Label Encoding age_group:")
print(" (Ordered: younger to older)")

age_mapping = {'18-25': 0, '26-35': 1, '36-45': 2, '46-55': 3, '55+': 4}
data['age_group_encoded'] = data['age_group'].map(age_mapping)

print(f" '18-25' → 0")
print(f" '26-35' → 1")
print(f" '36-45' → 2")
print(f" '46-55' → 3")
print(f" '55+' → 4")
print()

# Method B: One-Hot Encoding (for unordered categories)
print("One-Hot Encoding income_bracket:")
print(" (Creates separate column for each category)")

income_dummies = pd.get_dummies(data['income_bracket'], prefix='income')
data = pd.concat([data, income_dummies], axis=1)

print(" Original column: ['Low', 'Mid', 'High', ...]")
print(" New columns:")
for col in income_dummies.columns:
    print(f" {col}: 1 if that bracket, 0 otherwise")
print()

```

```

# =====

# TECHNIQUE 7: Scaling Numerical Features
# =====

print("=" * 50)
print("TECHNIQUE 7: SCALING")
print("=" * 50)

# Problem: income ranges from $20k-$200k, age from 18-69
# Models like these to be on similar scales!

print("Before scaling:")
print(f" Income: ${data['annual_income'].min():.0f} to ${data['annual_income'].max():.0f}")
print(f" Age: {data['applicant_age'].min()} to {data['applicant_age'].max()}")
print(f" Credit: {data['credit_score'].min()} to {data['credit_score'].max()}")
print()

# StandardScaler: Converts to mean=0, std=1
# (Each value becomes: "how many standard deviations from the mean")
scaler = StandardScaler()

numerical_features = ['annual_income', 'loan_amount', 'credit_score', 'employment_years']

# Fit and transform
data_scaled = data.copy()
data_scaled[numerical_features] = scaler.fit_transform(data[numerical_features])

print("After scaling (StandardScaler):")
print(f" All features now centered around 0")
print(f" Income scaled range: {data_scaled['annual_income'].min():.2f} to {data_scaled['annual_income'].max():.2f}")
print(f" Age scaled range: {data_scaled['applicant_age'].min():.2f} to {data_scaled['applicant_age'].max():.2f}")
print()

# =====
# FEATURE ENGINEERING SUMMARY
# =====

print("=" * 50)
print("FEATURE ENGINEERING SUMMARY")
print("=" * 50)

print(f"Started with: {len(['application_id', 'applicant_age', 'annual_income', 'loan_amount', 'credit_score', 'loan_to_income_ratio', 'age_group', 'income_bracket', 'total_existing_debt', 'stability_score', 'age_squared', 'income_credit_interaction', 'high_debt_ratio', 'is_young', 'excellent_credit'])} features")
print(f"Now have: {data.shape[1]} features")
print()

print("New features created:")
new_features = ['debt_to_income_ratio', 'loan_to_income', 'age_group', 'income_bracket',
                'total_existing_debt', 'stability_score', 'age_squared',
                'income_credit_interaction', 'high_debt_ratio', 'is_young', 'excellent_credit']

```

```

for i, feat in enumerate(new_features, 1):
    print(f" {i}. {feat}")

print()
print("✓ Features are now more informative and model-ready!")

# Save engineered dataset
data.to_csv('loan_data_engineered.csv', index=False)
print("\n✓ Saved to 'loan_data_engineered.csv'")

```

Key Insights:

Why each feature matters:

- `debt_to_income_ratio`: Banks' #1 metric for loan risk
 - `age_group`: Different age groups have different default patterns
 - `stability_score`: Combines multiple stability indicators
 - `income_credit_interaction`: Captures "super safe" and "super risky" extremes
 - **Scaling**: Prevents features with large numbers from dominating
-

6. Data Splitting

WHY: Why split the data?

Imagine studying for a test:

- You practice with sample problems
- Then take the real test with NEW problems

If the test had the SAME problems you practiced with, your score wouldn't show your true understanding. You might have just memorized answers.

Machine learning needs the same approach:

- **Training set**: Problems to learn from
- **Validation set**: Problems to tune your approach
- **Test set**: Final exam with completely new problems

Golden Rule: Never let the model see the test set until final evaluation!

HOW: How do we split data?

Standard split ratios:

- **70-15-15:** 70% training, 15% validation, 15% test
- **80-10-10:** More training data
- **60-20-20:** More evaluation data

For time-series: Use chronological split (train on past, test on future)

For imbalanced data: Use stratified split (keeps class ratios)

DO IT: Real Example

python

```
import pandas as pd
from sklearn.model_selection import train_test_split

# Load engineered data
data = pd.read_csv('loan_data_engineered.csv')

print("=" * 50)
print("PREPARING FOR SPLITTING")
print("=" * 50)

# =====
# STEP 1: Separate Features and Target
# =====

print("\nStep 1: Separate X (features) and y (target)")
print("-" * 50)

# Target variable (what we want to predict)
y = data['defaulted']

# Features (what we use to predict)
# Remove: ID (not useful), target (that's what we're predicting),
# and text categories (already encoded)
columns_to_drop = ['application_id', 'defaulted', 'age_group', 'income_bracket']
X = data.drop(columns=columns_to_drop)

print(f"Features (X): {X.shape[1]} columns")
print(f"Target (y): {len(y)} values")
print(f"\nTarget distribution:")
print(f" Class 0 (paid back): {(y==0).sum()} ({(y==0).sum()/len(y):.1%})")
print(f" Class 1 (defaulted): {(y==1).sum()} ({(y==1).sum()/len(y):.1%})")
print()

# =====
# STEP 2: First Split (Train + Val vs Test)
# =====

print("=" * 50)
print("STEP 2: CREATE TRAIN+VAL AND TEST SETS")
print("=" * 50)

# Split: 85% for train+validation, 15% for test
# stratify=y ensures both sets have same default ratio
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y,
    test_size=0.15,    # 15% for test
    random_state=42,   # For reproducibility
```

```

stratify=y      # Keep same default ratio
)

print(f"Total data: {len(X)}")
print(f"Train+Val: {len(X_temp)} (85%)")
print(f"Test: {len(X_test)} (15%)")
print()

print("Test set class distribution:")
print(f" Class 0: {(y_test==0).sum()} ({(y_test==0).sum()/len(y_test):.1%})")
print(f" Class 1: {(y_test==1).sum()} ({(y_test==1).sum()/len(y_test):.1%})")
print(" ✓ Matches original distribution!")
print()

# =====
# STEP 3: Second Split (Train vs Validation)
# =====

print("=" * 50)
print("STEP 3: SPLIT TRAIN+VAL INTO TRAIN AND VAL")
print("=" * 50)

# Split the 85% into train (70% of total) and val (15% of total)
# 70/85 ≈ 0.823
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp,
    test_size=0.176, # To get 15% of original
    random_state=42,
    stratify=y_temp
)

print(f"Final split:")
print(f" Training: {len(X_train)} ({len(X_train)/len(X):.1%})")
print(f" Validation: {len(X_val)} ({len(X_val)/len(X):.1%})")
print(f" Test: {len(X_test)} ({len(X_test)/len(X):.1%})")
print()

# =====
# STEP 4: Verify the Split
# =====

print("=" * 50)
print("VERIFICATION")
print("=" * 50)

print("\nClass distribution across all sets:")
print(f" Training: {(y_train==1).sum()}/{len(y_train)} = {(y_train==1).sum()/len(y_train):.1%} defaults")
print(f" Validation: {(y_val==1).sum()}/{len(y_val)} = {(y_val==1).sum()/len(y_val):.1%} defaults")
print(f" Test: {(y_test==1).sum()}/{len(y_test)} = {(y_test==1).sum()/len(y_test):.1%} defaults")
print(" ✓ All sets have similar default rates!")

```

```

print("All sets have similar column rates")
print()

print("Feature overlap check:")
print(f" Training features: {list(X_train.columns[:5])}...")
print(f" Test features: {list(X_test.columns[:5])}...")
print(" ✓ Same features in all sets!")
print()

# =====
# STEP 5: Save the Splits
# =====

print("=" * 50)
print("SAVING SPLITS")
print("=" * 50)

# Save for later use
X_train.to_csv('X_train.csv', index=False)
X_val.to_csv('X_val.csv', index=False)
X_test.to_csv('X_test.csv', index=False)
y_train.to_csv('y_train.csv', index=False)
y_val.to_csv('y_val.csv', index=False)
y_test.to_csv('y_test.csv', index=False)

print("✓ Saved all splits to CSV files")
print()

# =====
# IMPORTANT RULES
# =====

print("=" * 50)
print("CRITICAL RULES FOR SPLIT DATA")
print("=" * 50)

print("""
1. NEVER train on validation or test data
  ✗ Bad: model.fit(X_test, y_test)
  ✓ Good: model.fit(X_train, y_train)

2. NEVER use test set for tuning
  ✗ Bad: Try 10 models, pick best on test set
  ✓ Good: Try 10 models, pick best on validation, then test once

3. NEVER fit scalers/encoders on test data
  ✗ Bad: scaler.fit(X_test)
  ✓ Good: scaler.fit(X_train), then scaler.transform(X_test)

4. Test set is sacred - use it ONLY ONCE at the very end

```

5. If test results are bad, don't go back and retrain
(That would be "peeking" at the test set)
""")

Sample Output:

```
=====
PREPARING FOR SPLITTING
=====
```

Step 1: Separate X (features) and y (target)

Features (X): 20 columns

Target (y): 995 values

Target distribution:

Class 0 (paid back): 846 (85.0%)

Class 1 (defaulted): 149 (15.0%)

```
=====
STEP 2: CREATE TRAIN+VAL AND TEST SETS
=====
```

Total data: 995

Train+Val: 845 (85%)

Test: 150 (15%)

Test set class distribution:

Class 0: 127 (84.7%)

Class 1: 23 (15.3%)

✓ Matches original distribution!

```
=====
STEP 3: SPLIT TRAIN+VAL INTO TRAIN AND VAL
=====
```

Final split:

Training: 696 (69.9%)

Validation: 149 (15.0%)

Test: 150 (15.1%)

Why this matters:

- **Training (70%):** Largest set - model learns patterns here
- **Validation (15%):** Tune hyperparameters, compare models
- **Test (15%):** Final, unbiased evaluation
- **Stratification:** Ensures each set represents the real-world ratio

7. Model Selection

WHY: Why choose different models?

Imagine you're learning to catch a ball:

- **Simple approach:** "Ball comes, put hand up"
- **Complex approach:** Calculate trajectory, wind speed, spin rate, air resistance...

Sometimes simple works fine. Sometimes you need complex. Same with ML models:

- **Simple models** (Logistic Regression): Fast, interpretable, work with less data
- **Complex models** (Neural Networks): Powerful, need lots of data, harder to explain

HOW: How do we select models?

Consider:

1. **Data size:** Small data → simple models; Big data → complex models OK
2. **Interpretability:** Need to explain? → Linear models; Don't need to explain? → Black boxes OK
3. **Speed:** Need instant predictions? → Fast models; Can wait? → Slow models OK
4. **Problem type:** Classification vs Regression vs Clustering

DO IT: Real Example

python

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
import time
```

Load our split data

```
X_train = pd.read_csv('X_train.csv')
y_train = pd.read_csv('y_train.csv').values.ravel()
X_val = pd.read_csv('X_val.csv')
y_val = pd.read_csv('y_val.csv').values.ravel()
```

```
print("=" * 50)
print("MODEL SELECTION OVERVIEW")
print("=" * 50)
```

```
print("\nOur problem:")
print(" Type: Binary Classification (default: yes/no)")
print(" Data: 696 training samples, 20 features")
print(" Goal: Predict loan defaults")
print(" Constraint: Bank needs interpretable model")
print()
```

```
# =====
# MODEL 1: Logistic Regression
# =====
```

```
print("=" * 50)
print("MODEL 1: LOGISTIC REGRESSION")
print("=" * 50)
```

```
print("\nWhat it is:")
print(" A linear model that outputs probabilities")
print(" Formula:  $P(\text{default}) = 1 / (1 + e^{-(b_0 + b_1 \cdot x_1 + b_2 \cdot x_2 + \dots)})$ ")
print()
```

```
print("Pros:")
print(" ✓ Very interpretable (shows feature importance)")
print(" ✓ Fast to train and predict")
print(" ✓ Works well with limited data")
print(" ✓ Outputs probabilities")
print()
```

```
print("\nCons:")
```

```

print("Cons:")
print(" ✗ Assumes linear relationships")
print(" ✗ Can't capture complex patterns")
print(" ✗ Struggles with feature interactions")
print()

print("Best for:")
print(" • When you need to explain decisions")
print(" • Linear-ish relationships")
print(" • Baseline model")
print()

# Train it
start_time = time.time()
lr_model = LogisticRegression(random_state=42, max_iter=1000)
lr_model.fit(X_train, y_train)
train_time = time.time() - start_time

print(f"Training time: {train_time:.4f} seconds")
print(f"Model size: ~{X_train.shape[1] * 8} bytes (very small!)")
print()

# =====
# MODEL 2: Decision Tree
# =====

print("=" * 50)
print("MODEL 2: DECISION TREE")
print("=" * 50)

print("\nWhat it is:")
print(" Makes decisions using if-then rules")
print(" Example: IF credit_score < 600 AND income < 50k THEN high_risk")
print()

print("Pros:")
print(" ✓ Very interpretable (can visualize as flowchart)")
print(" ✓ Handles non-linear relationships")
print(" ✓ No need to scale features")
print(" ✓ Captures interactions automatically")
print()

print("Cons:")
print(" ✗ Easily overfits (memorizes training data)")
print(" ✗ Unstable (small data changes = big tree changes)")
print(" ✗ Not great with continuous numerical features")
print()

print("Best for:")
print(" • When you need simple explanations")

```

```

print(" • When you need simple explanations ")
print(" • Categorical features")
print(" • As building block for ensembles")
print()

start_time = time.time()
dt_model = DecisionTreeClassifier(random_state=42, max_depth=5)
dt_model.fit(X_train, y_train)
train_time = time.time() - start_time

print(f"Training time: {train_time:.4f} seconds")
print(f"Tree depth: 5 levels (prevents overfitting)")
print()

# =====
# MODEL 3: Random Forest
# =====

print("=" * 50)
print("MODEL 3: RANDOM FOREST")
print("=" * 50)

print("\nWhat it is:")
print(" Builds many decision trees and averages their votes")
print(" Each tree trained on random subset of data and features")
print()

print("Pros:")
print(" ✓ Very accurate")
print(" ✓ Handles overfitting better than single tree")
print(" ✓ Works with mixed feature types")
print(" ✓ Provides feature importance")
print(" ✓ Robust to outliers")
print()

print("Cons:")
print(" ✗ Slower to train (building many trees)")
print(" ✗ Larger model size")
print(" ✗ Less interpretable than single tree")
print(" ✗ Can be overkill for simple problems")
print()

print("Best for:")
print(" • When you want high accuracy")
print(" • Tabular data (like ours)")
print(" • When you have enough data")
print()

start_time = time.time()
rf_model = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)

```



```

rf_model.fit(X_train, y_train)
train_time = time.time() - start_time

print(f"Training time: {train_time:.4f} seconds")
print(f"Number of trees: 100")
print(f"Model size: Larger (stores 100 trees)")
print()

# =====
# MODEL 4: Gradient Boosting
# =====

print("=" * 50)
print("MODEL 4: GRADIENT BOOSTING")
print("=" * 50)

print("\nWhat it is:")
print("  Builds trees sequentially, each fixing errors of previous")
print("  Like a team where each member corrects others' mistakes")
print()

print("Pros:")
print("  ✓ Often highest accuracy")
print("  ✓ Great with tabular data")
print("  ✓ Handles mixed feature types")
print("  ✓ Feature importance available")
print()

print("Cons:")
print("  ✗ Slow to train (sequential, can't parallelize)")
print("  ✗ Easy to overfit if not tuned properly")
print("  ✗ More hyperparameters to tune")
print("  ✗ Less interpretable")
print()

print("Best for:")
print("  • Kaggle competitions")
print("  • When accuracy is paramount")
print("  • Structured/tabular data")
print()

start_time = time.time()
gb_model = GradientBoostingClassifier(n_estimators=100, random_state=42)
gb_model.fit(X_train, y_train)
train_time = time.time() - start_time

print(f"Training time: {train_time:.4f} seconds (slowest so far)")
print()

```

```

# =====
# MODEL COMPARISON TABLE
# =====

print("=" * 70)
print("MODEL COMPARISON FOR OUR LOAN DEFAULT PROBLEM")
print("=" * 70)

print(f"\n{'Model':<20} {'Speed':<10} {'Accuracy':<12} {'Interpretability':<18} {'Best Use':<15}")
print("-" * 70)

models_info = [
    ("Logistic Regression", "★★★★★", "★★★☆☆", "★★★★★", "Baseline"),
    ("Decision Tree", "★★★★☆", "★★★☆☆", "★★★★★", "Simple rules"),
    ("Random Forest", "★★★☆☆", "★★★★★", "★★★☆☆", "High accuracy"),
    ("Gradient Boosting", "★★☆☆☆", "★★★★★", "★★☆☆☆", "Best accuracy"),
    ("SVM", "★★☆☆☆", "★★★★☆", "★☆☆☆☆", "Complex boundaries"),
    ("K-Nearest Neighbors", "★★★★☆", "★★★☆☆", "★★☆☆☆", "Simple concept"),
]

for model, speed, acc, interp, use in models_info:
    print(f"{model:<20} {speed:<10} {acc:<12} {interp:<18} {use:<15}")

print()

# =====
# RECOMMENDATION FOR OUR PROBLEM
# =====

print("=" * 50)
print("RECOMMENDATION")
print("=" * 50)

print("""
For our loan default prediction problem:

PRIMARY MODEL: Random Forest
Why:


- Need high accuracy (money at stake)
- Have enough data (696 samples)
- Bank can accept some black-box
- Still provides feature importance for insights



BACKUP MODEL: Logistic Regression
Why:


- If bank needs full interpretability
- For regulatory compliance
- As baseline comparison
- Faster for real-time scoring


""")

```

NEXT STEPS:

1. Train both models
2. Compare performance on validation set
3. Choose based on accuracy vs interpretability trade-off
4. Fine-tune the chosen model

"""

```
print("\n✓ Ready to train and compare models!")
```

Key Decision Factors:

1. **Business need:** Bank regulators often require explainable models → favor Logistic Regression
 2. **Data size:** 696 samples is moderate → Random Forest OK, Neural Networks might overfit
 3. **Real-time requirement:** 2 second limit → Rules out very slow models
 4. **Cost of errors:** False negatives (missing defaults) costly → Optimize recall
-

8. Model Training

WHY: Why train models?

Think of training like teaching:

- **Before training:** Model has no idea what patterns lead to defaults
- **During training:** Model sees thousands of examples, finds patterns
- **After training:** Model can make educated guesses on new applicants

The model adjusts internal parameters to minimize errors on training data.

HOW: How do we train?

Basic process:

1. **Initialize** model with random or default parameters
2. **Feed** training data (X_train, y_train)
3. **Model learns** patterns by minimizing error
4. **Validate** on separate validation set
5. **Repeat** if needed with different settings

DO IT: Real Example

python

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns
```

Load data

```
X_train = pd.read_csv('X_train.csv')
y_train = pd.read_csv('y_train.csv').values.ravel()
X_val = pd.read_csv('X_val.csv')
y_val = pd.read_csv('y_val.csv').values.ravel()
```

```
print("=" * 50)
print("MODEL TRAINING")
print("=" * 50)
print(f"\nTraining on: {len(X_train)} samples")
print(f"Validating on: {len(X_val)} samples")
print(f"Features: {X_train.shape[1]}")
print()
```

```
# =====
# TRAINING MODEL 1: Logistic Regression
# =====
```

```
print("=" * 50)
print("TRAINING: LOGISTIC REGRESSION")
print("=" * 50)
```

```
print("\nInitializing model...")
lr_model = LogisticRegression(
    random_state=42,
    max_iter=1000,
    class_weight='balanced' # Handle imbalanced classes
)
```

```
print("✓ Model initialized")
print(" Parameters:")
print(f"   max_iter: 1000 (maximum iterations)")
print(f"   class_weight: 'balanced' (gives more weight to minority class)")
print()
```

```
print("Training model...")
lr_model.fit(X_train, y_train)
print("✓ Training complete!")
```

```

print()

# Make predictions
print("Making predictions on validation set...")
lr_predictions = lr_model.predict(X_val)
lr_probabilities = lr_model.predict_proba(X_val)[: , 1] # Probability of default
print("✓ Predictions generated")
print()

# =====
# TRAINING MODEL 2: Random Forest
# =====

print("=" * 50)
print("TRAINING: RANDOM FOREST")
print("=" * 50)

print("\nInitializing model...")
rf_model = RandomForestClassifier(
    n_estimators=100,    # Number of trees
    max_depth=10,       # Maximum tree depth
    min_samples_split=10, # Minimum samples to split
    random_state=42,
    class_weight='balanced',
    n_jobs=-1           # Use all CPU cores
)

print("✓ Model initialized")
print(" Parameters:")
print(f"  n_estimators: 100 (build 100 trees)")
print(f"  max_depth: 10 (limit tree depth to avoid overfitting)")
print(f"  min_samples_split: 10 (need 10 samples to split node)")
print(f"  class_weight: 'balanced'")
print()

print("Training model (this may take a moment)...")
rf_model.fit(X_train, y_train)
print("✓ Training complete!")
print()

# Make predictions
print("Making predictions on validation set...")
rf_predictions = rf_model.predict(X_val)
rf_probabilities = rf_model.predict_proba(X_val)[: , 1]
print("✓ Predictions generated")
print()

# =====
# UNDERSTANDING PREDICTIONS
# =====

```

```

print("=" * 50)
print("UNDERSTANDING MODEL PREDICTIONS")
print("=" * 50)

print("\nLet's look at first 10 predictions:")
print(f"\n{'Actual':<10} {'LR Pred':<10} {'LR Prob':<12} {'RF Pred':<10} {'RF Prob':<12}")
print("-" * 60)

for i in range(10):
    actual = "Default" if y_val[i] == 1 else "Paid"
    lr_pred = "Default" if lr_predictions[i] == 1 else "Paid"
    rf_pred = "Default" if rf_predictions[i] == 1 else "Paid"

    print(f"{'actual':<10} {'lr_pred':<10} {'lr_probabilities[i]:>6.1%}    {'rf_pred':<10} {'rf_probabilities[i]:>6.1%}")

print()
print("What these numbers mean:")
print(" • Actual: What really happened")
print(" • Pred: Model's prediction (Default if probability > 50%)")
print(" • Prob: Model's confidence (0% = definitely won't default, 100% = definitely will)")
print()

# =====
# COMPARE TRAINING VS VALIDATION
# =====

print("=" * 50)
print("TRAINING VS VALIDATION PERFORMANCE")
print("=" * 50)

# Get training predictions
lr_train_pred = lr_model.predict(X_train)
rf_train_pred = rf_model.predict(X_train)

# Calculate accuracies
lr_train_acc = accuracy_score(y_train, lr_train_pred)
lr_val_acc = accuracy_score(y_val, lr_predictions)
rf_train_acc = accuracy_score(y_train, rf_train_pred)
rf_val_acc = accuracy_score(y_val, rf_predictions)

print("\nLogistic Regression:")
print(f" Training accuracy: {lr_train_acc:.1%}")
print(f" Validation accuracy: {lr_val_acc:.1%}")
print(f" Difference: {abs(lr_train_acc - lr_val_acc):.1%}")

if abs(lr_train_acc - lr_val_acc) < 0.05:
    print(" ✓ Small difference = Good! Not overfitting")
else:

```

```

print(" ⚠ Large difference = Might be overfitting")

print("\nRandom Forest:")
print(f" Training accuracy: {rf_train_acc:.1%}")
print(f" Validation accuracy: {rf_val_acc:.1%}")
print(f" Difference: {abs(rf_train_acc - rf_val_acc):.1%}")

if abs(rf_train_acc - rf_val_acc) < 0.05:
    print(" ✓ Small difference = Good! Not overfitting")
else:
    print(" ⚠ Large difference = Might be overfitting")

print()

# =====
# FEATURE IMPORTANCE
# =====

print("=" * 50)
print("FEATURE IMPORTANCE")
print("=" * 50)

print("\nRandom Forest: Top 10 Most Important Features")
print("(Higher = more important for predictions)\n")

# Get feature importance
feature_importance = pd.DataFrame({
    'feature': X_train.columns,
    'importance': rf_model.feature_importances_
}).sort_values('importance', ascending=False)

print(f"{'Feature':<30} {'Importance':<12} {'Bar'}")
print("-" * 70)

for idx, row in feature_importance.head(10).iterrows():
    bar = '█' * int(row['importance'] * 100)
    print(f"{row['feature']:<30} {row['importance']:>6.4f} {bar}")

print()
print("What this tells us:")
print(" • These features have the most influence on predictions")
print(" • Model relies heavily on these when making decisions")
print(" • Less important features could potentially be removed")
print()

# =====
# SAVE TRAINED MODELS
# =====

```

```

print("=" * 50)
print("SAVING MODELS")
print("=" * 50)

import joblib

joblib.dump(lr_model, 'logistic_regression_model.pkl')
joblib.dump(rf_model, 'random_forest_model.pkl')

print("✓ Models saved:")
print(" • logistic_regression_model.pkl")
print(" • random_forest_model.pkl")
print()

print("To load later:")
print(" loaded_model = joblib.load('random_forest_model.pkl')")
print()

print("✓ Training complete! Ready for evaluation.")

```

What Happened During Training:

Logistic Regression:

1. Started with random coefficients for each feature
2. Calculated predictions and compared to actual defaults
3. Adjusted coefficients to reduce errors
4. Repeated until error stopped decreasing

Random Forest:

1. Created 100 different trees
 2. Each tree trained on random subset of data and features
 3. Each tree learned different patterns
 4. Final prediction = majority vote of all trees
-

9. Model Evaluation

WHY: Why evaluate thoroughly?

Imagine a security system:

- **High sensitivity:** Catches all intruders, but many false alarms
- **High specificity:** Few false alarms, but misses some intruders

You need to know: Does this trade-off work for my situation?

ML evaluation answers:

- How accurate is the model?
- What types of errors does it make?
- Is it better than alternatives?
- Will it work in production?

HOW: How do we evaluate?

Key metrics for classification:

1. **Accuracy:** Overall correctness (can be misleading with imbalanced data)
2. **Precision:** Of those predicted as defaulters, how many actually defaulted?
3. **Recall:** Of actual defaulters, how many did we catch?
4. **F1-Score:** Balance between precision and recall
5. **Confusion Matrix:** Detailed breakdown of correct/incorrect predictions
6. **ROC-AUC:** Overall model performance across all thresholds

DO IT: Real Example

```
python
```

```
import pandas as pd
import numpy as np
import joblib
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             confusion_matrix, classification_report, roc_auc_score, roc_curve)
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Load models and data
```

```
lr_model = joblib.load('logistic_regression_model.pkl')
rf_model = joblib.load('random_forest_model.pkl')
```

```
X_val = pd.read_csv('X_val.csv')
y_val = pd.read_csv('y_val.csv').values.ravel()
```

```
# Get predictions
```

```
lr_pred = lr_model.predict(X_val)
lr_prob = lr_model.predict_proba(X_val)[:, 1]
```

```
rf_pred = rf_model.predict(X_val)
rf_prob = rf_model.predict_proba(X_val)[:, 1]
```

```
print("=" * 50)
print("MODEL EVALUATION")
print("=" * 50)
```

```
# =====
# METRIC 1: Confusion Matrix
# =====
```

```
print("\n" + "=" * 50)
print("CONFUSION MATRIX")
print("=" * 50)
```

```
print("\nWhat is a Confusion Matrix?")
print(" A table showing correct and incorrect predictions\n")
```

```
print("Logistic Regression:")
lr_cm = confusion_matrix(y_val, lr_pred)
print("\n      Predicted")
print("      No  Yes")
print(f"Actual No  {lr_cm[0,0]:>4} {lr_cm[0,1]:>4}")
print(f"Actual Yes {lr_cm[1,0]:>4} {lr_cm[1,1]:>4}")
```

```
print("\nBreakdown:")
print(f" True Negatives (TN): {lr_cm[0,0]} - Correctly predicted 'will pay back'")
print(f" False Positives (FP): {lr_cm[0,1]} - Wrongly predicted 'will default'")
print(f" False Negatives (FN): {lr_cm[1,0]} - Wrongly predicted 'will pay back'")
print(f" True Positives (TP): {lr_cm[1,1]} - Correctly predicted 'will default'")
```

```

print(f" False Negatives (FN): {lr_cm[1,0]} - Wrongly predicted 'will pay back'")
print(f" True Positives (TP): {lr_cm[1,1]} - Correctly predicted 'will default'")
print()

print("Random Forest:")
rf_cm = confusion_matrix(y_val, rf_pred)
print("\n      Predicted")
print("      No   Yes")
print(f"Actual No   {rf_cm[0,0]:>4} {rf_cm[0,1]:>4}")
print(f"Actual Yes   {rf_cm[1,0]:>4} {rf_cm[1,1]:>4}")
print()

# =====
# METRIC 2: Basic Metrics
# =====

print("=" * 50)
print("ACCURACY, PRECISION, RECALL, F1")
print("=" * 50)

print("\n" + "="*70)
print(f"{'Metric':<20} {'Logistic Reg':<20} {'Random Forest':<20}")
print("="*70)

# Calculate metrics
lr_acc = accuracy_score(y_val, lr_pred)
rf_acc = accuracy_score(y_val, rf_pred)
print(f"{'Accuracy':<20} {lr_acc:>6.1%}          {rf_acc:>6.1%}")

lr_prec = precision_score(y_val, lr_pred)
rf_prec = precision_score(y_val, rf_pred)
print(f"{'Precision':<20} {lr_prec:>6.1%}          {rf_prec:>6.1%}")

lr_rec = recall_score(y_val, lr_pred)
rf_rec = recall_score(y_val, rf_pred)
print(f"{'Recall':<20} {lr_rec:>6.1%}          {rf_rec:>6.1%}")

lr_f1 = f1_score(y_val, lr_pred)
rf_f1 = f1_score(y_val, rf_pred)
print(f"{'F1-Score':<20} {lr_f1:>6.1%}          {rf_f1:>6.1%}")

lr_auc = roc_auc_score(y_val, lr_prob)
rf_auc = roc_auc_score(y_val, rf_prob)
print(f"{'ROC-AUC':<20} {lr_auc:>6.1%}          {rf_auc:>6.1%}")

print("="*70)
print()

# =====
# UNDERSTANDING EACH METRIC

```

```

# =====

print("=" * 50)
print("WHAT DO THESE METRICS MEAN?")
print("=" * 50)

print(f"\n1. ACCURACY: {rf_acc:.1%}")
print(" • Overall correctness")
print(" • Formula: (TP + TN) / Total")
print(f" • Means: {rf_acc:.1%} of all predictions were correct")
print(" • Problem: Can be misleading with imbalanced data!")
print(f" • Example: If 85% don't default, always predicting 'no default'")
print("          gives 85% accuracy but useless model!")
print()

print(f"2. PRECISION: {rf_prec:.1%}")
print(" • Of predicted defaults, how many were correct?")
print(" • Formula: TP / (TP + FP)")
print(f" • Means: When we say 'will default', we're right {rf_prec:.1%} of time")
print(" • Important when: False alarms are costly")
print(" • For bank: High precision = don't reject too many good customers")
print()

print(f"3. RECALL (Sensitivity): {rf_rec:.1%}")
print(" • Of actual defaults, how many did we catch?")
print(" • Formula: TP / (TP + FN)")
print(f" • Means: We catch {rf_rec:.1%} of actual defaulters")
print(" • Important when: Missing positives is costly")
print(" • For bank: High recall = don't approve too many bad loans")
print()

print(f"4. F1-SCORE: {rf_f1:.1%}")
print(" • Balanced average of precision and recall")
print(" • Formula: 2 * (Precision * Recall) / (Precision + Recall)")
print(" • Useful when: You want balance between precision and recall")
print()

print(f"5. ROC-AUC: {rf_auc:.1%}")
print(" • Measures performance across all probability thresholds")
print(" • Range: 0.5 (random guessing) to 1.0 (perfect)")
print(f" • {rf_auc:.1%} means model is very good at distinguishing classes")
print()

# =====
# BUSINESS INTERPRETATION
# =====

print("=" * 50)
print("BUSINESS INTERPRETATION (Random Forest)")

```

```

print("=" * 50)

print("\nFor our bank's loan approval system:\n")

# Calculate business metrics
TP = rf_cm[1,1] # Caught defaulters
FN = rf_cm[1,0] # Missed defaulters
FP = rf_cm[0,1] # Wrongly rejected
TN = rf_cm[0,0] # Correctly approved

# Assume average loan is $30,000, default loses 70% of loan
avg_loan = 30000
default_loss_rate = 0.70
loss_per_default = avg_loan * default_loss_rate # $21,000

# Calculate financial impact
caught_defaults_saved = TP * loss_per_default
missed_defaults_cost = FN * loss_per_default
false_alarm_opportunity_cost = FP * (avg_loan * 0.05) # Lost profit

print(f"Out of {len(y_val)} loan applications:")
print(f" • Correctly approved: {TN} good customers")
print(f" • Correctly rejected: {TP} bad customers")
print(f" • Wrongly rejected: {FP} good customers (false alarm)")
print(f" • Wrongly approved: {FN} bad customers (missed default)")
print()

print("Financial Impact:")
print(f" 💰 Saved by catching defaults: ${caught_defaults_saved:,.0f}")
print(f" 💸 Lost to missed defaults: ${missed_defaults_cost:,.0f}")
print(f" 📉 Lost profit from false rejections: ${false_alarm_opportunity_cost:,.0f}")
print(f" 📊 Net benefit: ${caught_defaults_saved - missed_defaults_cost - false_alarm_opportunity_cost:,.0f}")
print()

# =====
# MODEL COMPARISON
# =====

print("=" * 50)
print("WHICH MODEL IS BETTER?")
print("=" * 50)

print("\nBased on our business goals (from Phase 1):")
print(" Goal: Precision ≥ 75%, Recall ≥ 70%\n")

print("Logistic Regression:")
print(f" Precision: {lr_prec:.1%} {'✓' if lr_prec >= 0.75 else '✗'}")
print(f" Recall: {lr_rec:.1%} {'✓' if lr_rec >= 0.70 else '✗'}")
print(f" F1-Score: {lr_f1:.1%}")

```

```

print()

print("Random Forest:")
print(f" Precision: {rf_prec:.1%} {'✓' if rf_prec >= 0.75 else '✗'}")
print(f" Recall: {rf_rec:.1%} {'✓' if rf_rec >= 0.70 else '✗'}")
print(f" F1-Score: {rf_f1:.1%}")
print()

if rf_f1 > lr_f1:
    print("🏆 WINNER: Random Forest")
    print(" • Higher F1-score")
    print(" • Better balance of precision and recall")
    print(" • Recommended for deployment")
else:
    print("🏆 WINNER: Logistic Regression")
    print(" • Better interpretability")
    print(" • Meets business requirements")
    print(" • Simpler to maintain")

print()

# =====
# DETAILED CLASSIFICATION REPORT
# =====

print("=" * 50)
print("DETAILED CLASSIFICATION REPORT")
print("=" * 50)

print("\nRandom Forest Performance:")
print(classification_report(y_val, rf_pred,
                           target_names=['Will Pay Back', 'Will Default']))

print("\nSupport: Number of actual samples in each class")
print("Macro avg: Simple average across classes")
print("Weighted avg: Average weighted by number of samples")
print()

print("✓ Evaluation complete! Model is ready for optimization.")

```

Sample Output:

=====

CONFUSION MATRIX

=====

Logistic Regression:

	Predicted	
	No	Yes
Actual No	108	18
Actual Yes	8	15

Random Forest:

	Predicted	
	No	Yes
Actual No	115	11
Actual Yes	5	18

=====

ACCURACY, PRECISION, RECALL, F1

=====

Metric	Logistic Reg	Random Forest
=====		
Accuracy	82.6%	89.3%
Precision	45.5%	62.1%
Recall	65.2%	78.3%
F1-Score	53.6%	69.2%
ROC-AUC	85.3%	91.7%
=====		

BUSINESS INTERPRETATION (Random Forest)

Financial Impact:

- 💰 Saved by catching defaults: \$378,000
- 📉 Lost to missed defaults: \$105,000
- 📊 Lost profit from false rejections: \$16,500
- 📈 Net benefit: \$256,500

Key Insights:

- Random Forest outperforms Logistic Regression on all metrics
- 78.3% recall = catching most defaulters
- 62.1% precision = acceptable false alarm rate
- Net benefit of \$256,500 on validation set alone
- Model meets business requirements

10. Model Optimization

WHY: Why optimize models?

Imagine tuning a radio:

- Default settings: You hear the station but with static
- Optimized settings: Crystal clear sound

Models have "knobs" (hyperparameters) that control their behavior. Default settings are rarely optimal. Optimization finds the best combination.

HOW: How do we optimize?

Methods:

1. **Grid Search:** Try every combination (thorough but slow)
2. **Random Search:** Try random combinations (faster, often good enough)
3. **Bayesian Optimization:** Smart search based on previous results
4. **Manual tuning:** Based on understanding

DO IT: Real Example


```
python
```

```
import pandas as pd
import numpy as np
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import make_scorer, f1_score, classification_report
import joblib
```

```
# Load data
```

```
X_train = pd.read_csv('X_train.csv')
y_train = pd.read_csv('y_train.csv').values.ravel()
X_val = pd.read_csv('X_val.csv')
y_val = pd.read_csv('y_val.csv').values.ravel()
```

```
print("=" * 50)
print("MODEL OPTIMIZATION")
print("=" * 50)
```

```
# =====
# STEP 1: Understanding Hyperparameters
# =====
```

```
print("\n" + "=" * 50)
print("UNDERSTANDING HYPERPARAMETERS")
print("=" * 50)
```

```
print("\nHyperparameters are settings that control how the model learns.")
print("Think of them as 'knobs' you can adjust.\n")
```

```
print("Key Random Forest Hyperparameters:")
print()
```

```
print("1. n_estimators (Number of trees)")
print(" • Current: 100")
print(" • What it does: More trees = more stable but slower")
print(" • Too few: Underfitting (poor performance)")
print(" • Too many: Diminishing returns + slow")
print(" • Will try: 50, 100, 200, 300")
print()
```

```
print("2. max_depth (Maximum tree depth)")
print(" • Current: 10")
print(" • What it does: Controls how deep each tree can grow")
print(" • Too shallow: Underfitting (misses patterns)")
print(" • Too deep: Overfitting (memorizes training data)")
print(" • Will try: 5, 10, 15, 20, None (unlimited)")
print()
```

```
print("3. min_samples_split (Minimum samples to split a node)")
```

```

print("3. min_samples_split (Min samples to split node)")
print("    • Current: 10")
print("    • What it does: Need this many samples to create a split")
print("    • Too low: Overfitting")
print("    • Too high: Underfitting")
print("    • Will try: 2, 5, 10, 20")
print()

print("4. min_samples_leaf (Min samples in leaf node)")
print("    • Current: 1 (default)")
print("    • What it does: Each final node must have this many samples")
print("    • Prevents tiny, overfitted leaves")
print("    • Will try: 1, 2, 4, 8")
print()

print("5. max_features (Features to consider per split)")
print("    • Current: 'sqrt' (square root of total)")
print("    • What it does: Adds randomness, prevents correlation")
print("    • Will try: 'sqrt', 'log2', None (all features)")
print()

# =====
# STEP 2: Define Search Space
# =====

print("=" * 50)
print("DEFINING SEARCH SPACE")
print("=" * 50)

param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [5, 10, 15, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2']
}

total_combinations = (len(param_grid['n_estimators']) *
                      len(param_grid['max_depth']) *
                      len(param_grid['min_samples_split']) *
                      len(param_grid['min_samples_leaf']) *
                      len(param_grid['max_features']))

print(f"\nTotal combinations to try: {total_combinations}")
print("With 5-fold cross-validation: " + str(total_combinations * 5) + " model trainings!")
print()

# =====
# STEP 3: Randomized Search (Faster Approach)
# =====

```

```
print("=" * 50)
print("RUNNING RANDOMIZED SEARCH")
print("=" * 50)

print("\nRandomized Search tries random combinations.")
print("Much faster than Grid Search, often finds good solutions.\n")
```

Define model

```
rf_base = RandomForestClassifier(
    random_state=42,
    class_weight='balanced',
    n_jobs=-1
)
```

Define scoring

```
scorer = make_scorer(f1_score)
```

Set up randomized search

```
random_search = RandomizedSearchCV(
    estimator=rf_base,
    param_distributions=param_grid,
    n_iter=20,          # Try 20 random combinations
    cv=5,              # 5-fold cross-validation
    scoring=scorer,
    verbose=1,
    random_state=42,
    n_jobs=-1
)
```

```
print("Starting search (this will take a few minutes)...")
print("Testing 20 random combinations with 5-fold CV...")
print()
```

```
random_search.fit(X_train, y_train)
```

```
print("\n✓ Search complete!")
print()
```

```
# =====
```

STEP 4: Analyze Results

```
# =====
```

```
print("=" * 50)
print("OPTIMIZATION RESULTS")
print("=" * 50)
```

```
print("\nBest hyperparameters found:")
for param, value in random_search.best_params_.items():
```

```

print(f" {param}: {value}")

print(f"\nBest cross-validation F1-score: {random_search.best_score_:.1%}")
print()

# Get best model
best_model = random_search.best_estimator_

# Compare with default model
print("Performance comparison:")
print()

# Load original model
original_model = joblib.load('random_forest_model.pkl')

# Test both on validation set
original_pred = original_model.predict(X_val)
optimized_pred = best_model.predict(X_val)

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

print(f"{'Metric':<15} {'Original':<12} {'Optimized':<12} {'Improvement'}")
print("-" * 55)

orig_acc = accuracy_score(y_val, original_pred)
opt_acc = accuracy_score(y_val, optimized_pred)
print(f"{'Accuracy':<15} {orig_acc:>6.1%} {opt_acc:>6.1%} {opt_acc-orig_acc:>+6.1%}")

orig_prec = precision_score(y_val, original_pred)
opt_prec = precision_score(y_val, optimized_pred)
print(f"{'Precision':<15} {orig_prec:>6.1%} {opt_prec:>6.1%} {opt_prec-orig_prec:>+6.1%}")

orig_rec = recall_score(y_val, original_pred)
opt_rec = recall_score(y_val, optimized_pred)
print(f"{'Recall':<15} {orig_rec:>6.1%} {opt_rec:>6.1%} {opt_rec-orig_rec:>+6.1%}")

orig_f1 = f1_score(y_val, original_pred)
opt_f1 = f1_score(y_val, optimized_pred)
print(f"{'F1-Score':<15} {orig_f1:>6.1%} {opt_f1:>6.1%} {opt_f1-orig_f1:>+6.1%}")

print()

# =====
# STEP 5: Cross-Validation Results
# =====

print("=" * 50)
print("CROSS-VALIDATION INSIGHTS")
print("=" * 50)

```

```

print("\nWhat is Cross-Validation?")
print(" Instead of one train/val split, we make 5 different splits")
print(" Train on 4 parts, test on 1 part, rotate 5 times")
print(" This gives more reliable performance estimates\n")

# Get CV results
cv_results = pd.DataFrame(random_search.cv_results_)

# Top 5 parameter combinations
top_5 = cv_results.nlargest(5, 'mean_test_score')[
    ['params', 'mean_test_score', 'std_test_score']
]

print("Top 5 parameter combinations:")
print()
for idx, (i, row) in enumerate(top_5.iterrows(), 1):
    print(f"{idx}. F1-Score: {row['mean_test_score']:.1%} ( $\pm$ {row['std_test_score']:.1%})")
    print(f" Parameters: {row['params']}")
    print()

# =====
# STEP 6: Save Optimized Model
# =====

print("=" * 50)
print("SAVING OPTIMIZED MODEL")
print("=" * 50)

joblib.dump(best_model, 'random_forest_optimized.pkl')
print("✓ Saved as 'random_forest_optimized.pkl'")
print()

# =====
# FINAL RECOMMENDATIONS
# =====

print("=" * 50)
print("OPTIMIZATION SUMMARY")
print("=" * 50)

print(f"\nOriginal F1-Score: {orig_f1:.1%}")
print(f"Optimized F1-Score: {opt_f1:.1%}")
improvement = ((opt_f1 - orig_f1) / orig_f1) * 100
print(f"Improvement: {improvement:.1f}%")
print()

if opt_f1 > orig_f1:
    print("✓ Optimization successful! Use optimized model.")
else:

```

```
print("⚠ Original model performed better. Keep investigating.")
```

```
print("\nNext steps:")
```

```
print(" 1. Test optimized model on test set (final evaluation)")
```

```
print(" 2. Analyze any remaining errors")
```

```
print(" 3. Consider ensemble methods if needed")
```

```
print(" 4. Prepare for deployment")
```

What We Learned:

Optimization Impact:

- Systematic search finds better hyperparameters than defaults
- Cross-validation gives reliable performance estimates
- Trade-off: More trees = better performance but slower predictions
- Optimal settings depend on your specific data

Key Decisions:

- Used Randomized Search (faster than Grid Search)
 - Optimized for F1-score (balances precision and recall)
 - 5-fold CV (good balance of reliability and speed)
 - Saved best model for deployment
-

11. Model Deployment

WHY: Why deploy models?

A model sitting on your laptop helps nobody. Deployment means:

- Making predictions available to users/systems
- Integrating with existing business processes
- Enabling real-world impact

For our bank: Loan officers need instant default risk scores when reviewing applications.

HOW: How do we deploy?

Common deployment approaches:

1. **Batch predictions:** Run on scheduled batches of data
2. **Real-time API:** Respond to individual requests instantly
3. **Embedded:** Build model into application
4. **Edge deployment:** Run on user devices

We'll build a REST API (most common approach).

DO IT: Real Example

python

```
# =====
```

```
# FILE 1: app.py (Flask API)
```

```
# =====
```

```
"""
```

Save this as 'app.py'

This creates a web API that accepts loan applications and returns predictions

```
"""
```

```
from flask import Flask, request, jsonify
```

```
import joblib
```

```
import pandas as pd
```

```
import numpy as np
```

```
# Initialize Flask app
```

```
app = Flask(__name__)
```

```
# Load the trained model
```

```
print("Loading model...")
```

```
model = joblib.load('random_forest_optimized.pkl')
```

```
print("✓ Model loaded successfully!")
```

```
# Define expected features (must match training data)
```

```
EXPECTED_FEATURES = [
```

```
    'applicant_age', 'annual_income', 'loan_amount', 'credit_score',  
    'employment_years', 'has_mortgage', 'has_car_loan', 'debt_to_income_ratio',  
    'loan_to_income', 'total_existing_debt', 'stability_score', 'age_squared',  
    'income_credit_interaction', 'high_debt_ratio', 'is_young', 'excellent_credit',  
    'age_group_encoded', 'income_Low', 'income_Lower-Mid', 'income_Mid',  
    'income_Upper-Mid', 'income_High'
```

```
]
```

```
@app.route('/')
```

```
def home():
```

```
    """Home page with API information"""
```

```
    return """
```

```
<h1>Loan Default Prediction API</h1>
```

```
<h2>Endpoints:</h2>
```

```
<ul>
```

```
    <li><b>GET /</b> - This page</li>
```

```
    <li><b>POST /predict</b> - Make a prediction</li>
```

```
    <li><b>GET /health</b> - Check API health</li>
```

```
</ul>
```

```
<h2>Example Request to /predict:</h2>
```

```
<pre>
```

```
{
```



```

"applicant_age": 35,
"annual_income": 75000,
"loan_amount": 25000,
"credit_score": 720,
"employment_years": 8,
"has_mortgage": 1,
"has_car_loan": 0
}
</pre>
"""

```

```
@app.route('/health', methods=['GET'])
```

```
def health():
```

```
    """Health check endpoint"""
```

```
    return jsonify({
```

```
        'status': 'healthy',
```

```
        'model_loaded': model is not None
```

```
    })
```

```
@app.route('/predict', methods=['POST'])
```

```
def predict():
```

```
    """
```

Main prediction endpoint

Receives loan application data, returns default prediction

```
    """
```

```
try:
```

```
    # Get data from request
```

```
    data = request.get_json()
```

```
    # Validate required fields
```

```
    required_fields = ['applicant_age', 'annual_income', 'loan_amount',
```

```
                       'credit_score', 'employment_years', 'has_mortgage',
```

```
                       'has_car_loan']
```

```
    missing = [f for f in required_fields if f not in data]
```

```
    if missing:
```

```
        return jsonify({
```

```
            'error': f'Missing required fields: {missing}'
```

```
        }), 400
```

```
    # Engineer features (same as training)
```

```
    features = engineer_features(data)
```

```
    # Create DataFrame with correct column order
```

```
    input_df = pd.DataFrame([features], columns=EXPECTED_FEATURES)
```

```
    # Make prediction
```

```
    prediction = model.predict(input_df)[0]
```

```
    probability = model.predict_proba(input_df)[0][1]
```

```
probability = model.predict_proba(input_data)[0][1]
```

```
# Prepare response
```

```
response = {  
    'application': data,  
    'prediction': int(prediction),  
    'prediction_label': 'Will Default' if prediction == 1 else 'Will Pay Back',  
    'default_probability': float(probability),  
    'risk_level': get_risk_level(probability),  
    'recommendation': get_recommendation(probability)  
}
```

```
return jsonify(response)
```

```
except Exception as e:
```

```
    return jsonify({  
        'error': str(e)  
    }), 500
```

```
def engineer_features(data):
```

```
    """
```

```
    Engineer features from raw application data
```

```
    Must match exactly the feature engineering done during training!
```

```
    """
```

```
# Extract basic features
```

```
age = data['applicant_age']  
income = data['annual_income']  
loan = data['loan_amount']  
credit = data['credit_score']  
employment = data['employment_years']  
has_mortgage = data['has_mortgage']  
has_car_loan = data['has_car_loan']
```

```
# Create derived features
```

```
debt_to_income_ratio = loan / income  
loan_to_income = loan / (income / 12)  
total_existing_debt = has_mortgage + has_car_loan  
stability_score = (credit / 850) + (employment / 40)  
age_squared = age ** 2  
income_credit_interaction = income * credit  
high_debt_ratio = 1 if debt_to_income_ratio > 0.4 else 0  
is_young = 1 if age < 30 else 0  
excellent_credit = 1 if credit >= 750 else 0
```

```
# Age group encoding
```

```
if age <= 25:  
    age_group_encoded = 0  
elif age <= 35:  
    age_group_encoded = 1
```

```

elif age <= 45:
    age_group_encoded = 2
elif age <= 55:
    age_group_encoded = 3
else:
    age_group_encoded = 4

# Income bracket one-hot encoding
income_Low = 1 if income < 30000 else 0
income_LowerMid = 1 if 30000 <= income < 50000 else 0
income_Mid = 1 if 50000 <= income < 75000 else 0
income_UpperMid = 1 if 75000 <= income < 100000 else 0
income_High = 1 if income >= 100000 else 0

# Return all features in correct order
return [
    age, income, loan, credit, employment, has_mortgage, has_car_loan,
    debt_to_income_ratio, loan_to_income, total_existing_debt,
    stability_score, age_squared, income_credit_interaction,
    high_debt_ratio, is_young, excellent_credit, age_group_encoded,
    income_Low, income_LowerMid, income_Mid, income_UpperMid, income_High
]

def get_risk_level(probability):
    """Categorize risk based on probability"""
    if probability < 0.2:
        return 'Low Risk'
    elif probability < 0.5:
        return 'Medium Risk'
    elif probability < 0.7:
        return 'High Risk'
    else:
        return 'Very High Risk'

def get_recommendation(probability):
    """Provide recommendation based on probability"""
    if probability < 0.3:
        return 'Approve - Low default risk'
    elif probability < 0.5:
        return 'Approve with conditions - Moderate risk'
    elif probability < 0.7:
        return 'Review manually - High risk'
    else:
        return 'Deny - Very high default risk'

if __name__ == '__main__':
    print("\n" + "="*50)
    print("LOAN DEFAULT PREDICTION API")
    print("="*50)
    print("\n - Create a new user - ")

```

```

print("\nStarting server...")
print("API will be available at: http://localhost:5000")
print("\nPress CTRL+C to stop\n")

app.run(debug=True, host='0.0.0.0', port=5000)

# =====
# FILE 2: test_api.py (Test Script)
# =====

"""
Save this as 'test_api.py'
This tests the deployed API with example loan applications
"""

import requests
import json

# API endpoint
API_URL = 'http://localhost:5000/predict'

print("=" * 50)
print("TESTING LOAN DEFAULT PREDICTION API")
print("=" * 50)

# Test Case 1: Low-risk applicant
print("\n" + "=" * 50)
print("TEST 1: Low-Risk Applicant")
print("=" * 50)

low_risk = {
    "applicant_age": 45,
    "annual_income": 120000,
    "loan_amount": 20000,
    "credit_score": 780,
    "employment_years": 15,
    "has_mortgage": 1,
    "has_car_loan": 0
}

print("\nApplication Details:")
for key, value in low_risk.items():
    print(f" {key}: {value}")

response = requests.post(API_URL, json=low_risk)
result = response.json()

print(f"\nPrediction: {result['prediction_label']}")
print(f"Default Probability: {result['default_probability']:.1%}")
print(f"Risk Level: {result['risk_level']}")

```

```
print(f"Risk Level: {result['risk_level']}")
print(f"Recommendation: {result['recommendation']}")

# Test Case 2: High-risk applicant
print("\n" + "=" * 50)
print("TEST 2: High-Risk Applicant")
print("=" * 50)

high_risk = {
    "applicant_age": 23,
    "annual_income": 35000,
    "loan_amount": 40000,
    "credit_score": 580,
    "employment_years": 2,
    "has_mortgage": 0,
    "has_car_loan": 1
}

print("\nApplication Details:")
for key, value in high_risk.items():
    print(f" {key}: {value}")

response = requests.post(API_URL, json=high_risk)
result = response.json()

print(f"\nPrediction: {result['prediction_label']}")
print(f"Default Probability: {result['default_probability']:.1%}")
print(f"Risk Level: {result['risk_level']}")
print(f"Recommendation: {result['recommendation']}")

# Test Case 3: Medium-risk applicant
print("\n" + "=" * 50)
print("TEST 3: Medium-Risk Applicant")
print("=" * 50)

medium_risk = {
    "applicant_age": 32,
    "annual_income": 65000,
    "loan_amount": 28000,
    "credit_score": 670,
    "employment_years": 5,
    "has_mortgage": 1,
    "has_car_loan": 1
}

print("\nApplication Details:")
for key, value in medium_risk.items():
    print(f" {key}: {value}")

response = requests.post(API_URL, json=medium_risk)
```

```
result = response.json()

print(f"\nPrediction: {result['prediction_label']}")
print(f"Default Probability: {result['default_probability']:.1%}")
print(f"Risk Level: {result['risk_level']}")
print(f"Recommendation: {result['recommendation']}")
```

```
print("\n" + "=" * 50)
print("API TESTING COMPLETE")
print("=" * 50)
```

```
# =====
# DEPLOYMENT INSTRUCTIONS
# =====
```

```
"""
```

TO DEPLOY THIS API:

1. Install required packages:
pip install flask scikit-learn pandas numpy joblib
2. Ensure you have the trained model:
 - random_forest_optimized.pkl must be **in** same directory
3. Start the API server:
python app.py
4. In a new terminal, test the API:
python test_api.py
5. For production deployment:
 - Use gunicorn: gunicorn -w 4 -b 0.0.0.0:5000 app:app
 - Deploy to cloud: AWS, Google Cloud, Azure
 - Use Docker **for** containerization
 - Set up monitoring **and** logging
 - Implement authentication
 - Add rate limiting
 - Use HTTPS
6. Example production setup **with** Docker:

Dockerfile:

FROM python:3.9 WORKDIR /app COPY requirements.txt . RUN pip install -r requirements.txt
COPY app.py . COPY random_forest_optimized.pkl . CMD ["gunicorn", "-w", "4", "-b",
"0.0.0.0:5000", "app:app"]

```
Build: docker build -t loan-api .
Run: docker run -p 5000:5000 loan-api

"""

print("""
=" * 50)
print("DEPLOYMENT COMPLETE!")
print("=" * 50)

print("\nYour model is now:")
print("  ✓ Accessible via REST API")
print("  ✓ Can handle real-time predictions")
print("  ✓ Returns structured JSON responses")
print("  ✓ Includes risk levels and recommendations")
print()

print("Usage example (using curl):")
print("curl -X POST http://localhost:5000/predict \\\
-H \"Content-Type: application/json\" \\\
-d '{\
  \"applicant_age\": 35,\
  \"annual_income\": 75000,\
  \"loan_amount\": 25000,\
  \"credit_score\": 720,\
  \"employment_years\": 8,\
  \"has_mortgage\": 1,\
  \"has_car_loan\": 0\
}'\
")

print("\nNext steps for production:")
print(" 1. Add authentication (API keys, OAuth)")
print(" 2. Implement rate limiting")
print(" 3. Set up logging and monitoring")
print(" 4. Deploy to cloud platform")
print(" 5. Configure auto-scaling")
print(" 6. Set up CI/CD pipeline")
```

Key Deployment Concepts:

API Structure:

- **Endpoint:** URL where requests are sent (`/predict`)
- **Method:** HTTP POST (sending data)
- **Input:** JSON with loan application details
- **Output:** JSON with prediction and probabilities

Feature Engineering in Production:

- Must exactly match training feature engineering
- Any mismatch = wrong predictions
- This is why we create `engineer_features()` function

Error Handling:

- Validate input data
- Handle missing fields
- Catch model errors
- Return informative error messages

Scalability:

- Use multiple workers (`gunicorn -w 4`)
 - Implement caching for common requests
 - Monitor response times
 - Auto-scale based on traffic
-

12. Monitoring & Maintenance

WHY: Why monitor deployed models?

Imagine a weather forecast model:

- Trained on data from 2020
- Deployed in 2024
- But climate has changed!

Models degrade over time because:

- **Data drift:** Input data distribution changes
- **Concept drift:** Relationships between features and target change
- **Code issues:** Bugs, infrastructure problems
- **Business changes:** New products, policies, markets

Without monitoring, you won't know when your model stops working.

HOW: How do we monitor?

Track these metrics:

1. **Model performance:** Accuracy, precision, recall over time
2. **Data drift:** Are inputs different from training data?
3. **Prediction distribution:** Are predictions reasonable?
4. **System metrics:** Response time, errors, uptime
5. **Business metrics:** Actual defaults vs predictions

DO IT: Real Example

python

```
# =====  
# FILE: monitoring.py  
# =====  
  
"""  
Model monitoring system  
Tracks performance, drift, and system health  
"""  
  
import pandas as pd  
import numpy as np  
from datetime import datetime, timedelta  
import joblib  
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
class ModelMonitor:  
    """  
    Monitors deployed model performance and data quality  
    """  
  
    def __init__(self, model_path, training_data_path):  
        """Initialize monitor with model and training data statistics"""  
        self.model = joblib.load(model_path)  
  
        # Load training data to establish baseline  
        self.X_train = pd.read_csv(training_data_path)  
        self.training_stats = self.calculate_statistics(self.X_train)  
  
        # Initialize monitoring logs  
        self.performance_log = []  
        self.drift_log = []  
        self.prediction_log = []  
  
        print("✓ Monitor initialized")  
        print(f" Model loaded from: {model_path}")  
        print(f" Training baseline established")  
        print()  
  
    def calculate_statistics(self, data):  
        """Calculate statistical properties of data"""  
        stats = {}  
        for col in data.columns:  
            stats[col] = {  
                'mean': data[col].mean(),  
                'std': data[col].std(),  
                'min': data[col].min(),  
                'max': data[col].max(),  
                'count': data[col].count()
```

```

        'min': data[col].min(),
        'max': data[col].max(),
        'median': data[col].median()
    }
    return stats

def check_data_drift(self, new_data):
    """
    Detect if new data distribution differs from training data

    Uses statistical tests to compare distributions
    """
    print("=" * 50)
    print("DATA DRIFT DETECTION")
    print("=" * 50)
    print()

    new_stats = self.calculate_statistics(new_data)
    drift_detected = False
    drift_features = []

    for col in new_data.columns:
        # Calculate percentage change in mean
        train_mean = self.training_stats[col]['mean']
        new_mean = new_stats[col]['mean']

        if train_mean != 0:
            pct_change = abs((new_mean - train_mean) / train_mean) * 100
        else:
            pct_change = 0

        # Flag if mean shifted by more than 20%
        if pct_change > 20:
            drift_detected = True
            drift_features.append({
                'feature': col,
                'train_mean': train_mean,
                'new_mean': new_mean,
                'change_pct': pct_change
            })

    if drift_detected:
        print("⚠ DATA DRIFT DETECTED!")
        print("\nFeatures with significant drift:")
        for drift in drift_features:
            print(f"\n {drift['feature']}:")
            print(f"   Training mean: {drift['train_mean']:.2f}")
            print(f"   Current mean: {drift['new_mean']:.2f}")
            print(f"   Change: {drift['change_pct']:.1f}%")

```

```

print("\n ACTION REQUIRED:")
print("  - Investigate cause of drift")
print("  - Consider retraining model")
print("  - Review recent predictions")
else:
    print("✓ No significant data drift detected")

# Log drift check
self.drift_log.append({
    'timestamp': datetime.now(),
    'drift_detected': drift_detected,
    'drift_features': drift_features
})

print()
return drift_detected

def monitor_predictions(self, predictions, probabilities):
    """
    Monitor prediction distribution

    Checks if predictions look reasonable
    """
    print("=" * 50)
    print("PREDICTION MONITORING")
    print("=" * 50)
    print()

    default_rate = predictions.mean()
    avg_probability = probabilities.mean()

    print(f"Prediction Summary:")
    print(f"  Total predictions: {len(predictions)}")
    print(f"  Predicted defaults: {predictions.sum()} ({default_rate:.1%})")
    print(f"  Average default probability: {avg_probability:.1%}")
    print()

    # Check if default rate is unusual
    # Training data had 15% default rate
    expected_rate = 0.15

    if abs(default_rate - expected_rate) > 0.10: # More than 10% difference
        print("⚠ WARNING: Unusual default rate")
        print(f"  Expected: ~{expected_rate:.1%}")
        print(f"  Observed: {default_rate:.1%}")
        print("  This could indicate:")
        print("    - Change in applicant quality")
        print("    - Model degradation")
        print("    - Data quality issues")

```

```
else:
    print("✓ Default rate within expected range")
```

```
# Log predictions
```

```
self.prediction_log.append({
    'timestamp': datetime.now(),
    'num_predictions': len(predictions),
    'default_rate': default_rate,
    'avg_probability': avg_probability
})
```

```
print()
```

```
def evaluate_performance(self, X_new, y_true):
```

```
    """
```

```
    Evaluate model performance on new data with known outcomes
```

```
    This runs periodically as we get actual default outcomes
```

```
    """
```

```
print("=" * 50)
print("PERFORMANCE EVALUATION")
print("=" * 50)
print()
```

```
# Make predictions
```

```
y_pred = self.model.predict(X_new)
```

```
# Calculate metrics
```

```
accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred)
recall = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)
```

```
print("Current Performance:")
print(f" Accuracy: {accuracy:.1%}")
print(f" Precision: {precision:.1%}")
print(f" Recall: {recall:.1%}")
print(f" F1-Score: {f1:.1%}")
print()
```

```
# Compare to baseline (validation performance)
```

```
baseline_f1 = 0.692 # From our evaluation phase
```

```
if f1 < baseline_f1 - 0.05: # More than 5% degradation
```

```
    print("⚠ PERFORMANCE DEGRADATION DETECTED!")
    print(f" Baseline F1: {baseline_f1:.1%}")
    print(f" Current F1: {f1:.1%}")
    print(f" Degradation: {(baseline_f1 - f1):.1%}")
    print("\n RECOMMENDED ACTIONS:")
    print(f" 1. Check for data drift")
```

```

        print("    1. Check for data drift")
        print("    2. Retrain model with recent data")
        print("    3. Review feature engineering")
        print("    4. Consider model architecture changes")
    else:
        print("✓ Performance within acceptable range")

# Log performance
self.performance_log.append({
    'timestamp': datetime.now(),
    'accuracy': accuracy,
    'precision': precision,
    'recall': recall,
    'f1': f1
})

print()
return {'accuracy': accuracy, 'precision': precision,
        'recall': recall, 'f1': f1}

def generate_report(self):
    """Generate comprehensive monitoring report"""
    print("=" * 50)
    print("MONITORING REPORT")
    print("=" * 50)
    print(f"\nGenerated: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
    print()

# Performance over time
if self.performance_log:
    print("Performance Trend:")
    df_perf = pd.DataFrame(self.performance_log)
    print(df_perf.to_string(index=False))
    print()

# Drift incidents
drift_count = sum(1 for d in self.drift_log if d['drift_detected'])
print(f>Data Drift Incidents: {drift_count}")

# Prediction statistics
if self.prediction_log:
    df_pred = pd.DataFrame(self.prediction_log)
    print("\nPrediction Statistics:")
    print(f"    Total predictions made: {df_pred['num_predictions'].sum()}")
    print(f"    Average default rate: {df_pred['default_rate'].mean():.1%}")

print()

# =====
# USAGE EXAMPLE

```

```
# =====
```

```
print("=" * 50)
print("MODEL MONITORING SYSTEM")
print("=" * 50)
print()
```

```
# Initialize monitor
```

```
monitor = ModelMonitor(
    model_path='random_forest_optimized.pkl',
    training_data_path='X_train.csv'
)
```

```
# Simulate new incoming data
```

```
print("Simulating 1 week of production data...")
print()
```

```
# Load test set as "new" data
```

```
X_test = pd.read_csv('X_test.csv')
y_test = pd.read_csv('y_test.csv').values.ravel()
```

```
# Check for drift
```

```
monitor.check_data_drift(X_test)
```

```
# Make predictions and monitor
```

```
predictions = monitor.model.predict(X_test)
probabilities = monitor.model.predict_proba(X_test)[: , 1]
monitor.monitor_predictions(predictions, probabilities)
```

```
# Evaluate performance (after we get actual outcomes)
```

```
print("Evaluating performance on loans that matured...")
print()
monitor.evaluate_performance(X_test, y_test)
```

```
# Generate summary report
```

```
monitor.generate_report()
```

```
# =====
```

```
# ALERTING SYSTEM
```

```
# =====
```

```
class AlertSystem:
```

```
    """
```

```
    Sends alerts when issues are detected
```

```
    """
```

```
    @staticmethod
```

```
    def send_alert(alert_type, message):
```

```
        """
```

Send alert via email, Slack, PagerDuty, etc.

In production, this would integrate with:

- Email (SMTP)
- Slack (webhook)
- PagerDuty (API)
- AWS SNS
- Custom dashboard

```
"""
```

```
print("\n" + "!" * 50)
print(f"ALERT: {alert_type}")
print("!" * 50)
print(message)
print("!" * 50)
print()
```

In production:

- # - Send email to data science team*
- # - Post to Slack channel*
- # - Create incident ticket*
- # - Trigger automated response*

Example alert

```
AlertSystem.send_alert(
    alert_type="MODEL_PERFORMANCE_DEGRADATION",
    message="""
```

Model performance has degraded below acceptable threshold.

Current F1-Score: 65.3%

Baseline F1-Score: 69.2%

Degradation: 3.9%

Immediate Action Required:

1. Review recent predictions
2. Check for data quality issues
3. Schedule model retraining
4. Notify stakeholders

Incident ID: INC-2024-001

Severity: High

```
"""
```

```
)
```

```
print("\n" + "=" * 50)
print("MONITORING SYSTEM READY")
print("=" * 50)
print("""
```

In production, this system would:

1. Run continuously (24/7)
2. Check data quality on every prediction
3. Evaluate performance weekly
4. Generate daily/weekly reports
5. Send alerts for anomalies
6. Trigger automatic retraining when needed
7. Log all events for audit trail

Key Metrics to Track:

- Model latency (response time)
- Prediction accuracy
- Data drift indicators
- System uptime
- Error rates
- Business impact (actual \$ saved/lost)

Retraining Triggers:

- Performance drops > 5%
- Significant data drift detected
- New data patterns emerge
- Business requirements change
- Monthly scheduled retraining

"""

Sample Output:

=====

DATA DRIFT DETECTION

=====

⚠ DATA DRIFT DETECTED!

Features with significant drift:

annual_income:
Training mean: 109420.00
Current mean: 89250.00
Change: 18.4%

- ACTION REQUIRED:
- Investigate cause of drift
 - Consider retraining model
 - Review recent predictions

=====

PREDICTION MONITORING

=====

Prediction Summary:
Total predictions: 150
Predicted defaults: 23 (15.3%)
Average default probability: 18.7%

✓ Default rate within expected range

=====

PERFORMANCE EVALUATION

=====

Current Performance:
Accuracy: 89.3%
Precision: 62.1%
Recall: 78.3%
F1-Score: 69.2%

✓ Performance within acceptable range

Critical Monitoring Practices:

1. **Set Baselines:** Know what "normal" looks like
 2. **Define Thresholds:** When to alert (5% degradation? 10%?)
 3. **Automate:** Manual checking doesn't scale
 4. **Log Everything:** For debugging and auditing
 5. **Regular Retraining:** Monthly or quarterly, minimum
-

COMPLETE WORKFLOW SUMMARY

The Full Journey

1. PROBLEM DEFINITION (Week 1)
 - ↳ Define business goal, success metrics
2. DATA COLLECTION (Week 1-2)
 - ↳ Gather data from databases, APIs, files
3. EXPLORATORY DATA ANALYSIS (Week 2)
 - ↳ Understand data, find patterns, identify issues
4. DATA CLEANING (Week 2-3)
 - ↳ Handle missing values, remove duplicates, fix outliers
5. FEATURE ENGINEERING (Week 3)
 - ↳ Create new features, encode categories, scale
6. DATA SPLITTING (Week 3)
 - ↳ 70% train, 15% validation, 15% test
7. MODEL SELECTION (Week 4)
 - ↳ Choose algorithms based on problem requirements
8. MODEL TRAINING (Week 4)
 - ↳ Train models, compare performance
9. MODEL EVALUATION (Week 4-5)
 - ↳ Calculate metrics, confusion matrix, business impact
10. MODEL OPTIMIZATION (Week 5)
 - ↳ Tune hyperparameters, improve performance
11. MODEL DEPLOYMENT (Week 6)
 - ↳ Build API, deploy to production
12. MONITORING & MAINTENANCE (Ongoing)
 - ↳ Track performance, detect drift, retrain

Key Takeaways

1. It's Iterative

- You'll go back and forth between steps
- New insights lead to better features
- Failed experiments teach valuable lessons

2. Data Quality > Model Complexity

- Better data beats fancier algorithms
- Garbage in = garbage out
- Spend time on data cleaning and feature engineering

3. Business Context Matters

- Technical metrics (accuracy) $\neq$ business metrics (\$ saved)
- False posit